

# The Maze III (LeetCode 499)

© Authored by Arpit Raj, LNMIIT Jaipur | July 23, 2026

★★★★★★★★ Pattern: Dijkstra + Lexicographically Smallest Path

## 1. Problem Statement

We are navigating exactly the same maze mechanics as earlier iterations, but with distinct changes in rules and objectives. The grid parameters are:

- 0 = Empty space
- 1 = Wall

**The twist:** Aadya starts at the ball. Instead of a destination, there is now a **hole**. Unlike Maze I and Maze II, the ball falls into the hole *immediately* upon crossing it. It does not continue rolling until it hits a wall. The moment Aadya reaches the hole (H), the game ends.

**Goal:** We need to return the *lexicographically smallest path* among all shortest paths. If it is impossible to reach the hole, return "impossible".

## 2. Concept: Lexicographical Ordering

When multiple paths result in the same shortest distance, we must tie-break using "dictionary order" strings representing the directions taken.

"d" (Down) < "l" (Left) < "r" (Right) < "u" (Up)

### Example Scenario:

Suppose two paths reach cell (3, 4) with the same distance of 10.

- Path 1: "dr" (Down then Right)
- Path 2: "ld" (Left then Down)

In Maze II, the second path might just be ignored because the distance is not strictly shorter. However, because "dr" < "ld", we **must update** our answer to prefer the lexicographically smaller path!

### 3. The Algorithm Framework

---

#### Data Structures Needed

Instead of storing only the shortest distance in a `dist[] []` array, each cell must remember two things: *Best Distance* + *Best String (Path)*. Thus, we maintain:

- `dist[] []` initialized to infinity.
- `path[] []` initialized to a dummy high character (like "~").

#### Priority Queue Setup

The core of Dijkstra is the Priority Queue (Min-Heap). To factor in both distance and lexicographical path priority natively, the queue structure transforms:

| From Standard Dijkstra                             | To Maze III Dijkstra                                                   |
|----------------------------------------------------|------------------------------------------------------------------------|
| <code>pair&lt;int, pair&lt;int, int&gt;&gt;</code> | <code>pair&lt;int, pair&lt;string, pair&lt;int, int&gt;&gt;&gt;</code> |
| <code>{{distance, {row, col}}}</code>              | <code>{{distance, {path_string, {row, col}}}}</code>                   |

*Complexity:* Priority Queue operations take  $O(\log(nm))$ , resulting in an overall time complexity of  $O(nm \log(nm))$ .

#### The Rolling Logic Variation

The primary change in the rolling while loop is adding a break condition if the hole is found mid-roll.

```
while (can roll) {
    nr += dr;
    nc += dc;
    steps++;

    // New addition: Stop immediately if we hit the hole!
    if (nr == holeRow && nc == holeCol)
        break;
}
```

### 4. C++ Source Code (Solution)

---

Below is the perfectly formatted solution for LeetCode 499: The Maze III, keeping the original indentation and logic intact.

```

class Solution {
public:

    // direction + corresponding move character
    vector<pair<pair<int,int>,char>> dir={
        {{-1,0},'u'},
        {{0,-1},'l'},
        {{0,1},'r'},
        {{1,0},'d'}
    };

    string findShortestWay(vector<vector<int>>& maze, vector<int>& ball, vector<int>& hole) {

        int n=maze.size();
        int m=maze[0].size();

        vector<vector<int>> dist(n,vector<int>(m,INT_MAX));
        vector<vector<string>> path(n,vector<string>(m,"~"));

        priority_queue<
            pair<int,pair<string,pair<int,int>>>, //PIPSP pair int pair string pair
            vector<pair<int,pair<string,pair<int,int>>>>,
            greater<pair<int,pair<string,pair<int,int>>>> > pq;

        // initially Aadya is standing on the starting cell
        // distance travelled = 0
        // path taken = ""
        pq.push({0, {"", {ball[0], ball[1]}}});

        dist[ball[0]][ball[1]]=0;
        path[ball[0]][ball[1]]="";

        while(!pq.empty()){

            int currdist=pq.top().first; //PIPSP pair int pair string pair
            string currpath=pq.top().second.first;
            int r=pq.top().second.second.first;
            int c=pq.top().second.second.second;
            pq.pop();

            if(currdist>dist[r][c]){
                continue;
            }

            // same distance but lexicographically worse path kisi kaam ka ni
            if(currdist==dist[r][c] && currpath>path[r][c]){
                continue;
            }
        }
    }
};

```

```

}

//since priority queue always gives minimum distance first,
//the first time we found the hole it is our answer.
if(r==hole[0] && c==hole[1]){
    return currpath;
}

for(auto it:dir){

    int nr=r;
    int nc=c;
    int steps=0;

    // Keep rolling until next move becomes invalid.

    while(nr+it.first.first>=0 &&
          nr+it.first.first<n &&
          nc+it.first.second>=0 &&
          nc+it.first.second<m &&
          maze[nr+it.first.first][nc+it.first.second]==0){

        nr+=it.first.first;
        nc+=it.first.second;
        steps++;

        // If Aadya reaches the hole while rolling,
        // she immediately falls into it.
        // She DOES NOT continue till the wall or boundary
        if(nr==hole[0] && nc==hole[1]){
            break;
        }
    }

    string newpath=currpath+it.second;
    int newdist=currdist+steps;

    // Relaxation.
    // Update if
    // 1) shorter distance found
    // 2) same distance but lexicographically smaller path
    if(newdist<dist[nr][nc] ||
       (newdist==dist[nr][nc] && newpath<path[nr][nc])){

        dist[nr][nc]=newdist;
        path[nr][nc]=newpath;
    }
}

```

```
        pq.push({newdist, {newpath, {nr, nc}}});
    }
}
return "impossible";
};
```